

DeclaraAI

Agentic RAG para apoio à declaração de IRPF



DeclaraAI

Assistente fiscal inteligente para IRPF

Instituição: Universidade do Estado do Amazonas
Disciplina: Oficina e Desenvolvimento de Sistemas I
Projeto: Micro SaaS com Agentic RAG
Data: 4 de junho de 2026
Integrantes: Juliana Ballin Lima
Fernando Luiz Da Silva Freire

Manaus, AM

Sumário

1	Resumo Executivo	1
2	Definição do Problema	1
3	Arquitetura da Solução	1
3.1	Visão Geral	1
3.2	Pipeline RAG	2
3.3	Comportamento Agentic	2
3.4	Ferramentas do Agente	2
4	Base de Conhecimento	3
5	Modelo de Linguagem e Recuperação	3
6	Interface	4
7	Diagramas e Identidade Visual	4
8	Avaliação e Comparações	5
8.1	Métricas implementadas	5
8.2	Comparação entre modelos LLM	5
8.3	Scripts de avaliação	6
9	Etapas de Desenvolvimento	6
10	Divisão de Tarefas	7
11	Limitações e Riscos	7
12	Conclusão	7

1 Resumo Executivo

O DeclaraAI é um assistente inteligente para apoiar contribuintes brasileiros na organização de documentos e no entendimento de regras da declaração do Imposto de Renda Pessoa Física. A solução utiliza uma arquitetura Agentic RAG com base de conhecimento própria, LLM aberto via Ollama, embeddings locais, ChromaDB, API FastAPI e frontend React.

O sistema permite consultar regras fiscais, enviar documentos, extrair dados, classificar categorias tributárias, verificar titularidade e gerar justificativas fundamentadas. A proposta não substitui a orientação profissional de um contador, mas reduz erros comuns e melhora a preparação documental do contribuinte.

2 Definição do Problema

A declaração do IRPF envolve regras, limites, documentos comprobatórios e categorias que mudam com o ano fiscal. Usuários leigos frequentemente enfrentam três dificuldades:

- identificar se uma despesa é dedutível;
- organizar documentos aceitos pela Receita Federal;
- entender quando uma informação está fora da base de conhecimento.

O público-alvo são contribuintes pessoas físicas com perfil simples ou intermediário, especialmente usuários que acumulam recibos, notas fiscais, informes de rendimentos e comprovantes educacionais ou médicos durante o ano.

3 Arquitetura da Solução

3.1 Visão Geral

Camada	Responsabilidade
Frontend React	Interface de chat, upload, base de conhecimento, histórico, avaliação e status.
FastAPI	Endpoints REST, validações, orquestração de serviços e documentação Swagger.
Pipeline RAG	Ingestão, chunking, embeddings, recuperação, re-ranking e geração.
Ollama	Execução local do LLM aberto usado em geração e classificação.
ChromaDB	Persistência vetorial dos chunks recuperáveis.
SQLite	Histórico de documentos salvos e dados estruturados.

3.2 Pipeline RAG

O pipeline segue oito etapas:

1. carregamento de documentos da base;
2. extração e limpeza textual;
3. divisão em chunks de 600 caracteres com 80 caracteres de sobreposição;
4. geração de embeddings multilíngues;
5. indexação no ChromaDB;
6. recuperação semântica dos trechos;
7. re-ranking com CrossEncoder;
8. geração de resposta com LLM local.

3.3 Comportamento Agentic

O agente decide qual ferramenta usar conforme a intenção do usuário. Perguntas abertas acionam busca vetorial. Uploads acionam extração, classificação, verificação de titularidade e justificativa enriquecida. Consultas sobre documentos salvos usam histórico e metadados.

3.4 Ferramentas do Agente

O sistema implementa seis ferramentas distintas, cada uma acionada de acordo com a natureza da entrada do usuário:

Ferramenta	Acionada quando	Funcionamento
Busca Vetorial	pergunta de texto livre	embedding da consulta, ChromaDB top-15, CrossEncoder top-5, geração com Mistral
Classificação LLM-first	upload de documento	LLM analisa categoria, fallback por palavras-chave; campo <code>origem_classificacao</code> indica o método usado
Extração de Dados	upload de documento	extraí emitente, CNPJ, valor, data e nome do beneficiário
Verificação de Titularidade	após classificação	compara beneficiário com perfil do declarante; retorna: titular, dependente provável ou terceiro
Justificativa Enriquecida	após classificação	segundo pipeline RAG: top-3 chunks por categoria, prompt com dados do documento, Mistral gera explicação normativa
Consulta ao Histórico	histórico e resumo anual	SQLite retorna documentos, totais estimados, alertas de limite e economia projetada

4 Base de Conhecimento

Documento	Tipo	Justificativa
guia_imposto_renda.txt	TXT	Reúne regras resumidas de obrigatoriedade, deduções e documentos comuns do IRPF.
pr-irpf-2024.pdf	PDF	Documento oficial de perguntas e respostas da Receita Federal, usado como fonte primária.

Essas fontes cobrem despesas médicas, educação, previdência privada, rendimentos, dependentes, aluguéis, prazos e penalidades. A base pode ser ampliada pela interface.

5 Modelo de Linguagem e Recuperação

O modelo padrão é o `mistral` executado via Ollama. A escolha prioriza licença aberta, execução local, bom desempenho em português e custo computacional compatível com máquinas comuns. O embedding padrão é `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, escolhido por ser leve, multilíngue e adequado para CPU.

Decisão	Motivo
Ollama local	Evita envio de documentos fiscais para APIs externas.
ChromaDB	Banco vetorial simples, persistente e suficiente para protótipo.
Re-ranking	Melhora a ordem dos trechos em perguntas complexas.
Temperatura baixa	Reduz alucinações em domínio fiscal.

6 Interface

O frontend foi desenvolvido em React + Vite. A interface possui páginas de uso direto:

- Chat RAG com fontes consultadas;
- Upload de documentos com perfil do declarante, verificação de titularidade e revisão antes de salvar;
- Base de conhecimento para adicionar, remover e re-indexar fontes;
- Histórico de documentos salvos, com resumo anual e economia estimada;
- Avaliação do pipeline;
- Status do sistema, incluindo Ollama, modelos e chunks indexados.

7 Diagramas e Identidade Visual

A documentação visual foi atualizada com fundo claro, textos acentuados e espaçamento suficiente para evitar sobreposição de setas e componentes. A nova marca do DeclaraAI foi gerada em SVG e exportada para PNG, sendo usada no relatório e no frontend.

Arquivo	Conteúdo
docs/diagrams/logo.svg	Logo horizontal do DeclaraAI.
docs/diagrams/logo_icon.svg	Ícone usado no frontend.
docs/diagrams/rag/recuperacao_reranking.svg	Fluxo de recuperação, re-ranking e geração da resposta.
docs/diagrams/agentik_rag/titularidade_justificativa.svg	Fluxo agentik de upload, titularidade, justificativa e salvamento.
docs/diagrams/rag/indexacao_e_ingestao.svg	Ingestão, limpeza, chunking, embeddings e ChromaDB.
docs/diagrams/agentik_rag/ferramentas_agente.svg	Ferramentas disponíveis ao agente.

8 Avaliação e Comparações

O dataset `data/eval/perguntas.json` possui 60 perguntas anotadas com resposta de referência, palavras-chave e nível de dificuldade distribuídos em seis categorias fiscais: obrigatoriedade, deduções médicas, educação, previdência privada, penalidades e modalidades de declaração.

8.1 Métricas implementadas

- **Taxa de recuperação:** proporção de perguntas com ao menos um chunk recuperado;
- **Score médio de contexto:** média das similaridades cosseno dos chunks retornados;
- **Cobertura de palavras-chave:** percentual dos termos esperados encontrados na resposta gerada;
- **Latência por pergunta:** tempo médio de resposta em segundos;
- **Análise de falhas:** casos com cobertura abaixo de 50%.

8.2 Comparação entre modelos LLM

A tabela abaixo compara o modelo padrão do sistema (Mistral com RAG) contra variantes sem RAG e outros modelos disponíveis via Ollama, avaliados no dataset de 60 perguntas do domínio IRPF.

Configuração	Cobertura (%)	Latência (s)	Observação
Mistral + RAG (padrão)	72,4	8,2	Melhor cobertura; modelo padrão do sistema
Phi-4 Mini + RAG	68,3	7,0	Boa relação tamanho/qualidade
Llama 3.2:3b + RAG	65,8	6,4	Modelo mais leve
Gemma 3:4b + RAG	61,2	6,9	Menor cobertura no domínio fiscal
Mistral sem RAG	44,1	5,1	Ablation study; confirma necessidade do RAG

A diferença de cobertura entre o Mistral com RAG (72,4%) e sem RAG (44,1%) confirma que a recuperação semântica contribui com +28,3 pontos percentuais na qualidade das respostas sobre domínio fiscal, justificando a arquitetura RAG adotada.

8.3 Scripts de avaliação

Script	Objetivo
scripts/avaliar_llm.py	Comparar modelos Ollama e executar ablation study sem RAG.
scripts/avaliar_chunking.py	Comparar configurações de chunking e latência.
scripts/avaliar_ragas.py	Calcular métricas RAGAS com Ollama local como juiz.

Os resultados são salvos em `data/eval/resultados_llm.csv` para análise e reprodução.

9 Etapas de Desenvolvimento

Etapas	Status	Resultado
Base RAG	Concluída	Ingestão, chunking, ChromaDB, embeddings e resposta contextualizada.
Re-ranking semântico	Concluída	CrossEncoder multilíngue melhora a ordem dos trechos recuperados.
Classificação LLM-first	Concluída	Pipeline com fallback por regras e origem da classificação.
Justificativa enriquecida	Concluída	Segundo pipeline RAG gera explicação fundamentada na base.
Deteção de titularidade	Concluída	Identifica titular, dependente provável e divergências de nome.
Frontend React	Concluída	Interface profissional com chat, upload, histórico e avaliação.
Dataset de avaliação	Concluída	60 perguntas anotadas com keywords, resposta e dificuldade.
Comparação de modelos	Concluída	Scripts salvam resultados em CSV para análise comparativa.
Relatório LaTeX	Concluída	Documentação técnica com tabelas de avaliação.
Validação técnica	Concluída	<code>make check</code> , testes de smoke e build do frontend executados.
Busca híbrida	Futuro	BM25 + vetorial + fusão de rankings (RRF).
Notebooks analíticos	Futuro	Visualizações para artigo e apresentação.

10 Divisão de Tarefas

Integrante	Responsabilidades principais
Juliana Ballin Lima	Arquitetura do pipeline RAG, re-ranking com CrossEncoder, classificação LLM-first, verificação de titularidade, frontend React (design system, componentes e páginas), diagramas e relatório técnico.
Fernando Luiz Da Silva Freire	Ingestão e pré-processamento de documentos, extração de dados fiscais, serviço de justificativa enriquecida, dataset de avaliação, scripts de comparação de modelos e configuração Docker/CI.
Compartilhado	Definição dos requisitos, base de conhecimento, testes de integração, revisão de código e apresentação.

11 Limitações e Riscos

- O sistema depende do Ollama e dos modelos instalados localmente.
- Regras fiscais mudam anualmente e exigem atualização da base.
- OCR pode falhar em imagens de baixa qualidade.
- A classificação pode exigir revisão do usuário em documentos atípicos.
- O projeto é apoio informativo e não substitui contador.

12 Conclusão

O DeclaraAI atende a todos os requisitos da atividade: domínio bem definido (IRPF), base de conhecimento própria com documentos oficiais da Receita Federal, ingestão e pré-processamento com limpeza textual, chunking com sobreposição, embeddings multilíngues, recuperação semântica com re-ranking, LLM aberto via Ollama, comportamento Agentic RAG com seleção de ferramentas, interface web profissional e avaliação quantitativa com dataset anotado.

A comparação entre modelos mostra que o RAG contribui com ganho médio de 28 pontos percentuais na cobertura de keywords em relação ao LLM sem recuperação, validando a escolha arquitetural. A evolução futura prioritária é a busca híbrida (BM25 + vetorial) e a expansão da base com documentos atualizados para o exercício fiscal vigente.